Análise léxica e análise sintática em Java

01. Tokens

```java
import java.util.LinkedHashMap;
import java.util.Map;

public class Main {
    static final String ALUNO = "Guilherme Augusto dos Santos";
    static final String RA = "240273872";

    static final String ENTRADA = "resultado_ga = somaAnterior - valor / 72;";

    private static final Map<Character, String> SIMBOLOS = new LinkedHashMap<>();

    static {
        SIMBOLOS.put('=', "ATRIBUICAO");
        SIMBOLOS.put('+', "SOMA");
        SIMBOLOS.put('-', "SUBTRACAO");
        SIMBOLOS.put('*', "MULTIPLICACAO");
        SIMBOLOS.put('/', "DIVISAO");
        SIMBOLOS.put(';', "PONTO_E_VIRGULA");
        SIMBOLOS.put('(', "ABRE_PARENTESES");
        SIMBOLOS.put(')', "FECHA_PARENTESES");
    }
```

```
ALUNO: Guilherme Augusto dos Santos | RA: 240273872
ENTRADA: resultado_ga = somaAnterior - valor / 72;

LEXEMA                  TOKEN
-----------------------------------------
resultado_ga            IDENTIFICADOR
=                       ATRIBUICAO
somaAnterior            IDENTIFICADOR
-                       SUBTRACAO
valor                   IDENTIFICADOR
/                       DIVISAO
72                      INTEIRO
;                       PONTO_E_VIRGULA
```

02. Léxico

```
=== TESTE D - caractere inválido ===
ENTRADA: int valor# = 1;
LEXEMA                  CLASSE                  TOKEN
-----------------------------------------------------------------
int                     LETRA                   PALAVRA_RESERVADA
valor                   LETRA                   IDENTIFICADOR
#                       DESCONHECIDO            ERRO_LEXICO
=                       SIMBOLO                 ATRIBUICAO
1                       DIGITO                  INTEIRO
;                       SIMBOLO                 PONTO_E_VIRGULA
```

03. Erro compilador

```
Main.java:8: error: ';' expected
       int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
                           ^
Main.java:10: error: ')' expected
       if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
                      ^
Main.java:11: error: unclosed string literal
           System.out.println("Programa corrigido!); // CORRIGIR 3: feche o texto entre aspas
                              ^
3 errors
```

04. Programa corrigido

```
ALUNO: Guilherme_Augusto_dos_Santos | RA: 24027387-2
Programa corrigido!
Resultado: 30
```

05. Decrescente

```
ALUNO: Guilherme_Augusto_dos_Santos | RA: 24027387-2
ENTRADA: (7 + 3) * 4

ENTRANDO em <expr> | lookahead = (
  ENTRANDO em <term> | lookahead = (
    ENTRANDO em <factor> | lookahead = (
      Abrindo parenteses
      Novo lookahead: 7
      ENTRANDO em <expr> | lookahead = 7
        ENTRANDO em <term> | lookahead = 7
          ENTRANDO em <factor> | lookahead = 7
            Numero reconhecido: 7
            Novo lookahead: +
          SAINDO de <factor> | lookahead = +
        SAINDO de <term> | lookahead = +
        Encontrado operador +
        Novo lookahead: 3
        ENTRANDO em <term> | lookahead = 3
          ENTRANDO em <factor> | lookahead = 3
            Numero reconhecido: 3
            Novo lookahead: )
          SAINDO de <factor> | lookahead = )
        SAINDO de <term> | lookahead = )
      SAINDO de <expr> | lookahead = )
      Novo lookahead: *
      Fechando parenteses
    SAINDO de <factor> | lookahead = *
    Encontrado operador *
    Novo lookahead: 4
    ENTRANDO em <factor> | lookahead = 4
      Numero reconhecido: 4
      Novo lookahead: $FIM$
    SAINDO de <factor> | lookahead = $FIM$
  SAINDO de <term> | lookahead = $FIM$
SAINDO de <expr> | lookahead = $FIM$

ENTRADA ACEITA
RESULTADO DA EXPRESSAO: 40
```

06. Ascendente

```
ALUNO: Guilherme_Augusto_dos_Santos | RA: 24027387-2
ENTRADA: id+id*id

PILHA                          ENTRADA          ACAO
-------------------------------------------------------------------------
$ 0                            id+id*id$        SHIFT id -> estado 5
$ 0 id 5                       +id*id$          REDUCE F -> id
$ 0 F 3                        +id*id$          REDUCE T -> F
$ 0 T 2                        +id*id$          REDUCE E -> T
$ 0 E 1                        +id*id$          SHIFT + -> estado 6
$ 0 E 1 + 6                    id*id$           SHIFT id -> estado 5
$ 0 E 1 + 6 id 5               *id$             REDUCE F -> id
$ 0 E 1 + 6 F 3                *id$             REDUCE T -> F
$ 0 E 1 + 6 T 9                *id$             SHIFT * -> estado 7
$ 0 E 1 + 6 T 9 * 7            id$              SHIFT id -> estado 5
$ 0 E 1 + 6 T 9 * 7 id 5       $                REDUCE F -> id
$ 0 E 1 + 6 T 9 * 7 F 10       $                REDUCE T -> T * F
$ 0 E 1 + 6 T 9                $                REDUCE E -> E + T
$ 0 E 1                        $                ACCEPT
```